

Lab 6 - Preguntas y Reflexión Previa Programación

Preguntas:

1. Si arrancara *Uvicorn* con `127.0.0.1`, le estaría diciendo que solo escuche en la interfaz interna del contenedor. Esto significa que el servidor solo aceptaría peticiones que se originen desde dentro del mismo contenedor. Todo el tráfico que venga de fuera (incluido mi navegador web) sería ignorado.
En el contexto de redes, `0.0.0.0` es una forma de decir "escuchar en todas las interfaces de red disponibles" (hacer bind en todas las IPs).
Por ello, al usar `0.0.0.0`, le permito a *Uvicorn* aceptar el tráfico que entra a través de la interfaz de red virtual que *Docker* le ha asignado a ese contenedor. Solo así el mapeo de puertos (`ports: - "8000:8000"`) puede enrutar la petición desde la máquina host hacia el proceso que corre dentro de *Docker*.
2. Como comente en la pregunta anterior, dentro de un contenedor, la red está aislada. Cuando el frontend intenta ir a `localhost:8000`, está buscando en su propio puerto 8000, no en el del host ni el del backend.
Cuando levanto servicios con *docker-compose.yml*, *Docker* crea automáticamente una red virtual dedicada para esos contenedores, incluyendo además un servidor DNS embebido en esa red. Este servidor DNS asocia el nombre que le dimos al servicio en el YAML (en este caso, *backend*) con la dirección IP interna que *Docker* le acaba de asignar a ese contenedor. Por eso, al usar `http://backend:8000`, el DNS de *Docker* lo traduce correctamente y los contenedores pueden hablar entre sí usando sus nombres.
3. Dado que *Docker* construye las imágenes en "capas" (una por cada instrucción), si el contenido de una capa no cambia en la siguiente ejecución, *Docker* usa la versión guardada en caché para ir más rápido. Por ello si una capa se invalida (porque algo cambió), todas las capas posteriores también se invalidan y deben reconstruirse.
Si hiciera *COPY* . . antes del *RUN pip install*, cualquier cambio en el código (como editar un print en *main.py*) invalidará la capa de *COPY*. En consecuencia, la siguiente instrucción se ejecutará desde cero, descargando e instalando todas las librerías en cada pequeña modificación. El tiempo de construcción pasaría de milisegundos a varios minutos.
4. El riesgo principal de usar un archivo `.env` es el error humano: es muy fácil olvidar añadirlo al `.gitignore` (o hacer un `git add .` sin querer) y acabar publicando credenciales reales en un repositorio de código como GitHub.

Es por eso por lo que tenemos esas tres alternativas de producción:

- Variables de entorno del shell: Paso la variable justo al ejecutar el comando. Esto podría ser usado en entornos de desarrollo sencillos o en pipelines de CI/CD (como *GitHub Actions*) donde el runner inyecta los secretos de forma segura en tiempo de ejecución. El riesgo aquí es que el secreto puede quedar guardado en el historial de comandos de la terminal.
- *Docker Secrets* (Modo Swarm): La solución nativa de *Docker*. El secreto se cifra en la red y se monta dentro del contenedor como un archivo en memoria. Se puede usar cuando la infraestructura está basada puramente en un clúster de *Docker Swarm*.
- Servicios externos (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault): Servicios centralizados súper seguros. Nos conectaríamos al servicio mediante un rol o identidad temporal para recuperar el secreto en tiempo real. Es el método más común en entornos de producción empresariales, siendo la opción estándar en la industria.

Reflexión Previa Programación:

La clave para entender por qué necesitamos un health check separado a el arranque del contenedor yace en la diferencia entre "el contenedor está encendido" y "la aplicación está lista para trabajar".

Evaluyendo los tres escenarios propuestos podemos observar claramente esta diferencia:

- **Escenario 1:** En este caso, el uso de *depends_on* suele parecer que funciona bien, ya que el tiempo que tarda el frontend en arrancar su propio servidor suele ser suficiente para que el backend ya haya montado el modelo en memoria.
- **Escenario 2:** Es aquí donde se empiezan a observar las fragilidades del sistema. El contenedor del backend arranca inmediatamente, pero el proceso se queda bloqueado esperando a que la red descargue el archivo, pudiendo tardar entre 2 y 15 segundos. Durante este periodo, si el frontend lanzara una petición, recibirá errores de conexión de manera impredecible.
- **Escenario 3:** En modelos complejos (como LLMs), el contenedor estará "arriba" rápidamente, pero la API tardará mucho en poder procesar la primera inferencia. Esto causará fallos asegurados durante la fase de arranque, agotando el frontend sus intentos de reconexión antes siquiera de que esté listo el backend.

Para realizar estos cambios, necesitaremos asegurar tanto en el backend como en el *docker-compose.yml*. Comenzando por el backend, necesitaremos una ruta */health* la cual devuelva 200 si está preparado y 503 en caso contrario. Por suerte, esta ruta ya viene implementada en nuestro backend. Por otro lado, deberemos modificar *docker-compose.yml* para asegurar que el front espera a que el backend esté preparado para conectarse.

(El pseudocódigo para ambas partes están en el entregable)